
HikariForge

Documentación técnica de mi proyecto:
stack tecnológico, decisiones de diseño y manual de uso

Tienda de periféricos · API REST con Spring Boot + SPA
Autor: Antonio González

1. Descripción del proyecto

HikariForge es mi tienda online de periféricos informáticos (ratones, teclados, auriculares, alfombrillas, etc.). La he dividido en dos piezas independientes que se comunican mediante una API REST: un **backend** construido con Spring Boot, que expone los datos y la lógica de negocio, y un **frontend** de tipo SPA (Single Page Application) que consume esa API y se encarga de toda la interfaz.

He optado por esta separación —conocida como arquitectura desacoplada— porque es la forma habitual de construir aplicaciones web hoy en día. El backend no sabe nada de cómo se pinta la pantalla; solo responde con datos en formato JSON. Así puedo cambiar o reescribir el frontend sin tocar el servidor, y dejo la puerta abierta a tener varios clientes (web, móvil) consumiendo la misma API.

Funcionalidades principales

- Catálogo de productos con búsqueda, filtrado por categoría y paginación.
- Registro e inicio de sesión de usuarios con autenticación por token JWT.
- Roles diferenciados: el cliente navega y compra; el administrador gestiona el catálogo.
- Gestión de pedidos asociados a cada usuario.
- Documentación interactiva de la API mediante Swagger UI.

2. Visión general de la arquitectura

He estructurado el backend en una **arquitectura por capas**, donde cada capa tiene una única responsabilidad y solo se comunica con la inmediatamente inferior. Una petición del navegador recorre el siguiente camino:

```
Navegador (SPA)
  |
  v
Controller  -> recibe la petición HTTP y devuelve la respuesta
  |
  v
Service     -> contiene la logica de negocio
  |
  v
Repository  -> acceso a datos (Spring Data JPA)
  |
  v
PostgreSQL  -> base de datos
```

De forma transversal, **Spring Security** intercepta cada petición para validar el token JWT antes de dejarla pasar, y **Swagger UI** documenta todos los endpoints de forma automática. Separar las capas me hace el código más fácil de entender, probar y mantener: si mañana cambio de base de datos, solo tengo que tocar la capa de repositorio.

3. Stack tecnológico y justificación

Resumen de las tecnologías que he elegido. En las páginas siguientes explico el porqué de cada decisión.

Capa	Tecnología	Para qué
Lenguaje	Java 21 (LTS)	Lenguaje base del proyecto
Framework	Spring Boot 3.5	Estructura y autoconfiguración de la API
Build	Maven	Gestión de dependencias y empaquetado
Persistencia	Spring Data JPA + Hibernate	Mapear objetos Java a tablas
Base de datos	PostgreSQL 16	Almacenamiento relacional
Migraciones	Flyway	Versionar el esquema de la BD
Seguridad	Spring Security + JWT	Login, roles y protección de rutas
Documentación	springdoc (Swagger UI)	Probar la API desde el navegador
Contenedores	Docker + Compose	Levantar todo con un comando
Frontend	React + Vite	Interfaz que consume la API

Java 21 en lugar de versiones anteriores

Elegí Java 21 porque es una versión LTS (soporte a largo plazo), lo que me garantiza estabilidad y actualizaciones durante años. Frente a Java 8 u 11, aporta mejoras de rendimiento y sintaxis más moderna (records, switch mejorado), que aprovecho en los DTOs.

Spring Boot 3.5 en lugar de la 4.0

Aunque Spring Boot 4.0 ya existe, para este proyecto de aprendizaje preferí la línea 3.5: es madura, está plenamente soportada y cuenta con una enorme cantidad de tutoriales, documentación y respuestas en la comunidad. Eso me reduce muchísimo el tiempo perdido al resolver problemas. Migrar a la 4.x más adelante será un paso pequeño.

PostgreSQL en lugar de MySQL

A través de JPA el código es casi idéntico con cualquiera de las dos, pero me decidí por PostgreSQL por su mayor rigor con el estándar SQL, su mejor comportamiento ante escrituras concurrentes (carritos y pedidos simultáneos) y su soporte de columnas JSON, útil si más adelante guardo atributos variables por tipo de producto (DPI de un ratón, switches de un teclado). Es además la opción más habitual en backend moderno.

API REST + SPA en lugar de un monolito con plantillas

Separar backend y frontend me permite que cada parte evolucione por su cuenta, facilita tener varios clientes sobre la misma API y refleja cómo se construyen las aplicaciones reales hoy. El coste es algo más de configuración inicial (CORS, gestión del token), que asumo de buena gana porque me resulta muy formativo.

Spring Data JPA

Me evita escribir SQL repetitivo: con solo declarar una interfaz obtengo las operaciones CRUD y consultas derivadas del nombre de los métodos. Así me centro en el modelo de objetos, no en la fontanería del acceso a datos.

Flyway para las migraciones

En lugar de dejar que Hibernate cree o modifique las tablas (peligroso y poco controlable), uso Flyway para aplicar scripts SQL versionados (V1, V2...). Así el esquema queda documentado, es reproducible en cualquier máquina y forma parte de mi repositorio Git.

Spring Security con JWT

Al ser una API sin estado (stateless), no uso sesiones de servidor. Tras el login, el cliente recibe un token JWT firmado que envía en cada petición. El servidor lo valida sin necesidad de almacenar nada, lo que escala bien y encaja perfectamente con un frontend SPA.

DTOs y manejo global de errores

Uso DTOs para separar el contrato público de la API de las entidades internas (así no expongo datos sensibles como el hash de la contraseña). El manejador global traduce las excepciones a códigos HTTP coherentes (404, 400, 401) en un único sitio.

IDs como UUID en lugar de números secuenciales

Decidí que todos los identificadores fueran UUID (tokens) y no enteros consecutivos. PostgreSQL los genera con `gen_random_uuid()` y la API los expone como texto. Así no revelo cuántos registros tengo ni permito que nadie enumere o adivine recursos ajenos por la URL.

Docker y Docker Compose

Me permiten levantar la base de datos y la API con un solo comando, sin instalar PostgreSQL manualmente, y garantizan que el proyecto funcione igual en cualquier ordenador.

4. Modelo de datos

He modelado el dominio con cinco entidades principales y las siguientes relaciones:

- **Categoría:** agrupa los productos (Ratones, Teclados, Auriculares...).
- **Producto:** pertenece a una categoría (relación muchos-a-uno).
- **Usuario:** con email único, contraseña cifrada y un rol (CLIENTE o ADMIN).
- **Pedido:** pertenece a un usuario y agrupa varias líneas.
- **LineaPedido:** relaciona un pedido con un producto, su cantidad y el precio en el momento de la compra (se guarda aparte porque el precio del producto puede cambiar).

El precio lo almaceno con tipo decimal exacto (nunca coma flotante), una buena práctica obligada al trabajar con dinero.

Los identificadores de todas las tablas son **UUID** (tokens, no números secuenciales como 1, 2, 3). PostgreSQL los genera con `gen_random_uuid()` y la API los expone como texto. Uso tokens en lugar de enteros consecutivos para no revelar cuántos registros tengo y dificultar que se enumeren o

adivinen recursos ajenos por la URL.

5. Posibles mejoras y siguientes pasos

Ideas que tengo para hacer crecer el proyecto, ordenadas aproximadamente por valor frente a esfuerzo:

- **Tests:** añadir pruebas unitarias de los servicios (JUnit + Mockito) y de integración de los controladores. Es lo que más sube la calidad percibida del proyecto.
- **Carrito de la compra** y proceso de pedido completo (checkout), descontando stock.
- **Pasarela de pago** en modo de pruebas (Stripe sandbox) para simular el cobro.
- **Subida de imágenes** de producto (almacenamiento local o un servicio como Cloudinary).
- **Refresh tokens:** renovar el JWT sin obligar al usuario a volver a iniciar sesión.
- **Reseñas y valoraciones** de productos por parte de los clientes.
- **Integración continua (CI):** un workflow de GitHub Actions que compile y pase los tests en cada push.
- **Despliegue online** en una plataforma gratuita (Railway, Render o Fly.io) para poder enseñar el proyecto funcionando.
- **Panel de administración** en el frontend para gestionar productos y pedidos.

6. Manual de uso

En esta sección explico, paso a paso, cómo pongo en marcha mi aplicación y qué tiene que estar encendido en cada momento.

6.1. Qué necesito tener instalado

- **JDK 21** y **Maven** (para el backend), o únicamente **Docker Desktop** si quiero levantarlo todo en contenedores.
- **Node.js 18 o superior** (para el frontend).
- Opcional pero recomendable: un cliente gráfico de base de datos para ver los datos.

***Aclaración importante:** "Workbench" es la herramienta gráfica de **MySQL**. Como este proyecto usa **PostgreSQL**, el equivalente que instalo es **pgAdmin** (la herramienta oficial de PostgreSQL) o, como alternativa, **DBeaver** (sirve para cualquier base de datos). Es opcional: solo la uso para inspeccionar las tablas a mano; la aplicación funciona sin ella.*

6.2. Qué tiene que estar ENCENDIDO para que la app funcione

Mi aplicación tiene tres piezas. Para usarla por completo, las tres deben estar funcionando a la vez:

1. El servidor de base de datos (PostgreSQL)

Es lo primero que arranco. Sin la base de datos encendida, la API no arranca. La enciendo con Docker (lo habitual) o con una instalación local de PostgreSQL.

2. El backend (la API de Spring Boot)

Una vez la base de datos está lista, arranco la API. Al iniciarse, Flyway crea automáticamente las tablas y carga los datos de ejemplo.

3. El frontend (servidor de desarrollo de Vite)

La interfaz web. La arranco aparte y se comunica con la API. Para probar solo la API (con Swagger o Postman) no necesito tener el frontend encendido.

6.3. Forma recomendada: arrancar todo con Docker

Es la forma que uso normalmente: con un único comando levanto PostgreSQL y la API juntos. Desde la carpeta raíz del proyecto:

```
docker compose up --build
```

Cuando termina de arrancar tengo disponible:

- La API en `http://localhost:8080`
- Swagger UI en `http://localhost:8080/swagger-ui.html`

Para detenerlo pulso `Ctrl + C` y, si quiero eliminar los contenedores, ejecuto `docker compose down`.

6.4. Forma manual: la base de datos en Docker y la API con Maven

La uso mientras programo el backend, porque puedo reiniciar la API rápidamente.

Paso 1. Arranco solo la base de datos (queda en segundo plano):

```
docker compose up -d db
```

Paso 2. (Opcional) Abro pgAdmin o DBeaver y me conecto para comprobar que el servidor responde, con estos datos:

```
Host:      localhost
Puerto:   5432
Base:      hikariforge
Usuario:   hikariforge
Clave:     hikariforge
```

Paso 3. Arranco la API desde la carpeta backend:

```
cd backend
mvn spring-boot:run
```

En la consola veo cómo Flyway aplica las migraciones (V1, V2) y cómo Spring Boot arranca en el puerto 8080.

Paso 4. Compruebo que funciona abriendo Swagger UI en el navegador:

```
http://localhost:8080/swagger-ui.html
```

6.5. Cómo uso la API (registro, login y token)

La mayoría de endpoints están protegidos: necesito un token JWT para usarlos. Mi flujo es:

- 1 En Swagger, busco **POST /api/auth/register** y creo un usuario enviando un email, una contraseña (mínimo 6 caracteres) y un nombre. La respuesta incluye un token.
- 2 Si ya tengo usuario, uso **POST /api/auth/login** para obtener el token.
- 3 Copio el valor del campo token de la respuesta.
- 4 Pulso el botón **Authorize** (arriba a la derecha en Swagger), pego el token y confirmo. A partir de ahí, Swagger envía el token en todas las peticiones.
- 5 Ya puedo llamar a los endpoints protegidos. El catálogo (GET /api/productos) es público y no requiere token.

***Nota sobre el administrador:** los usuarios que registro son siempre CLIENTE. Para poder crear o borrar productos necesito un usuario ADMIN: le cambio el rol a un usuario directamente en la tabla usuario (con pgAdmin/DBeaver) poniendo el valor ADMIN en la columna rol.*

6.6. Arrancar el frontend

En otra terminal, dentro de la carpeta frontend (sigo las instrucciones de su README para crearlo la primera vez; en este proyecto uso pnpm, nunca npm):

```
cd frontend
pnpm install
pnpm dev
```

El frontend queda disponible en `http://localhost:5173`, que es el origen que tengo autorizado en la configuración de seguridad del backend (CORS).

6.7. Problemas frecuentes

La API no arranca y da error de conexión a la base de datos

Compruebo que PostgreSQL está encendido ANTES que la API (paso 1) y que el puerto 5432 no lo esté usando otro programa.

El navegador bloquea las peticiones del frontend (error CORS)

El frontend debe servirse en `http://localhost:5173`. Si uso otro puerto, lo añado a la lista de orígenes permitidos en la clase `SecurityConfig`.

Recibo 401 (no autorizado) en un endpoint

Falta el token o ha caducado. Vuelvo a hacer login y pulso Authorize de nuevo en Swagger.

Recibo 403 (prohibido) al crear un producto

Estoy autenticado pero no soy ADMIN. Cambio el rol de mi usuario a ADMIN en la base de datos.